

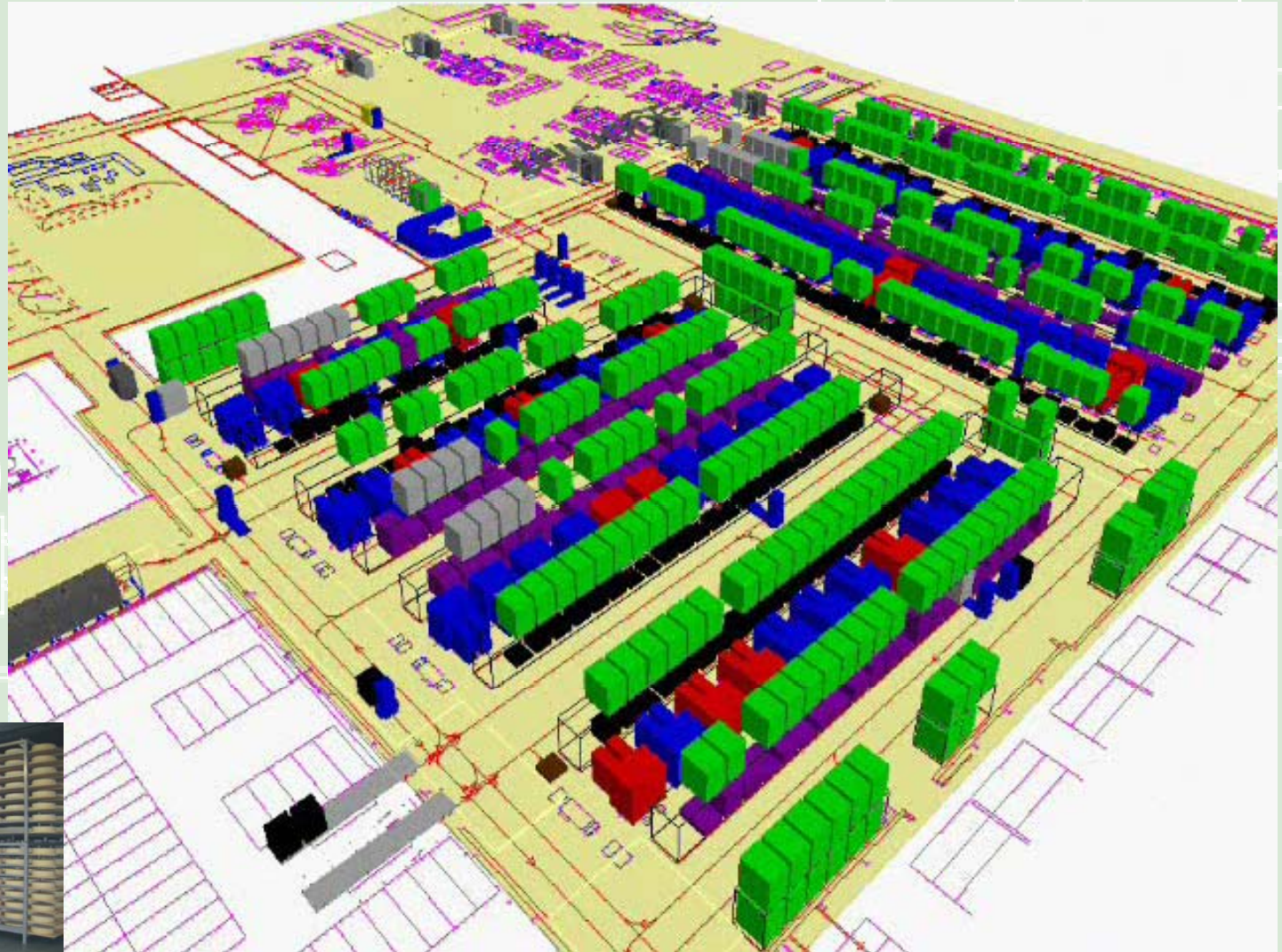
An Architecture-Centric Approach for Software Engineering with Situated Multiagent Systems

PhD Defense Danny Weyns
Katholieke Universiteit Leuven
October 11, 2006

Supervisors

Tom Holvoet & Pierre Verbaeten

A Challenging Application



Three Important Problem Characteristics

1. Dynamic and changing operating conditions
2. Inherent distribution of resources, locality of activity
3. Stakeholders have various – possibly conflicting – quality goals

Engineering such Software Systems

Structure of the software

1. Dynamic and changing operating conditions
2. Inherent distribution of resources, locality of activity
3. Stakeholders have various – possibly conflicting – quality goals

Engineering such Software Systems

Structure of the software

1. Dynamic and changing operating conditions
2. Inherent distribution of resources, locality of activity

3. Stakeholders have various – possibly conflicting – quality goals

Approach to engineer software

Structure of the Software

1. Dynamic and changing operating conditions
2. Inherent distribution of resources, locality of activity

Self-management: deal with dynamism and change autonomously

Decentralized control: system functionality results from cooperative subsystems



Situated Multiagent Systems

Approach to Engineer Software

3. Stakeholders have various – possibly conflicting – quality goals

Compel stakeholders to deal explicitly with (conflicting) quality goals

Architecture-Centric Software Engineering

Contribution of this Research

Problem characteristics:

1. Dynamism and change
2. Locality of activity
3. (Conflicting) quality goals

Engineering such systems

Architecture-centric
software engineering

Situated multiagent
systems

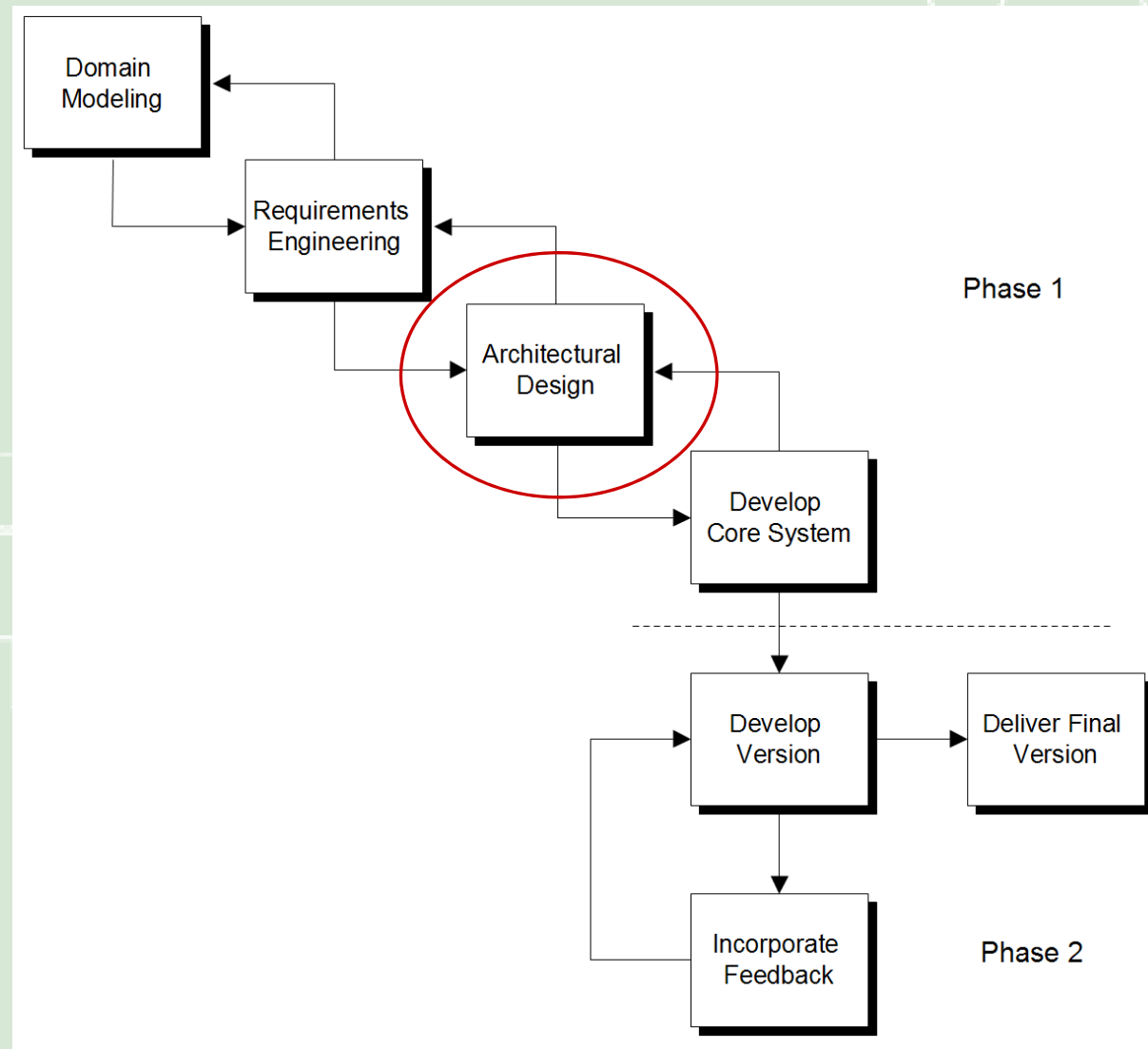
Expertise

Reference architecture for situated multiagent systems

Outline

- Architecture-Centric Software Engineering
 - Development Life-Cycle
 - Role of Reference Architecture
- Reference Architecture for Situated Multiagent Systems
- Conclusions

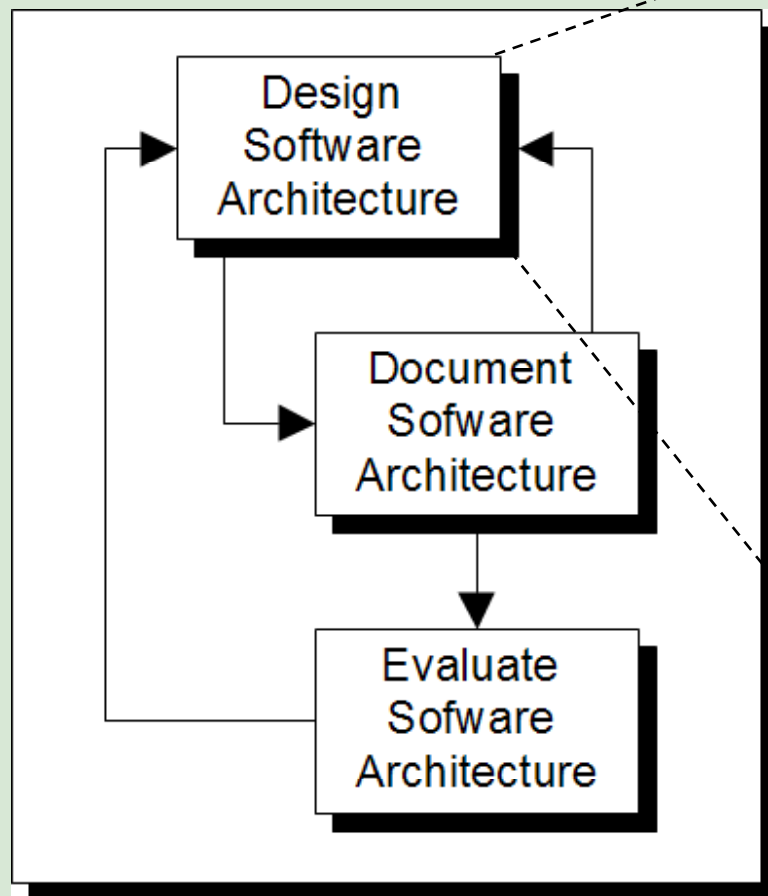
Architecture-Centric Software Engineering Development Life Cycle



Architecture-Centric Software Engineering

Role of Reference Architecture

Architectural Design



Design Software Architecture

Apply proven *architectural approaches* to achieve main quality requirements

Reference Architecture

“Best of best practices”

Integrated set of architectural patterns

Blueprint to develop new software architectures for systems with similar requirements

Outline

- Architecture-Centric Software Engineering
- Reference Architecture for Situated Multiagent Systems
 - Background
 - Environment as a First-Class Design Abstraction
 - Advanced Mechanisms for Adaptability
- Conclusions

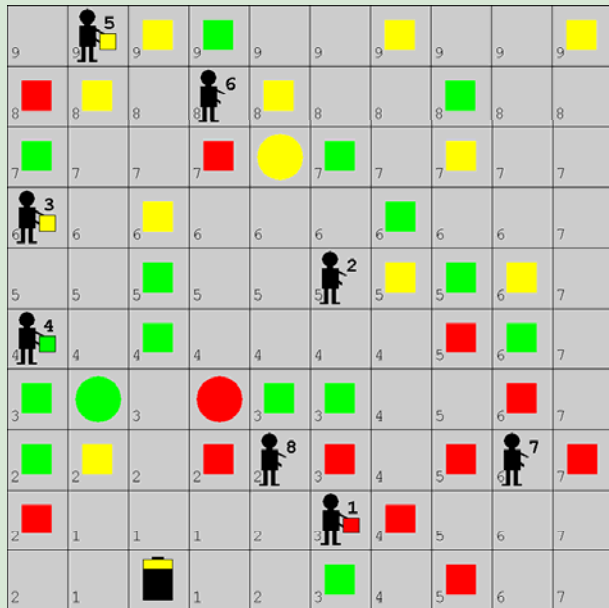
Reference Architecture

Background

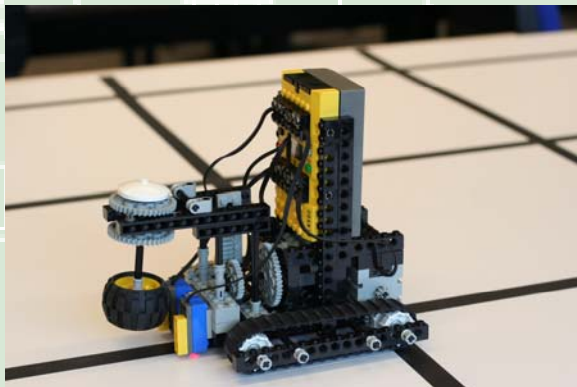
- Target domain
 - Dynamism and change
 - Locality of activity
 - Important quality goals: flexibility and openness
- Development process
 - Research and development of various applications
 - Derived common functions and structures
= building blocks of the reference architecture

Reference Architecture

Background



Some of the applications



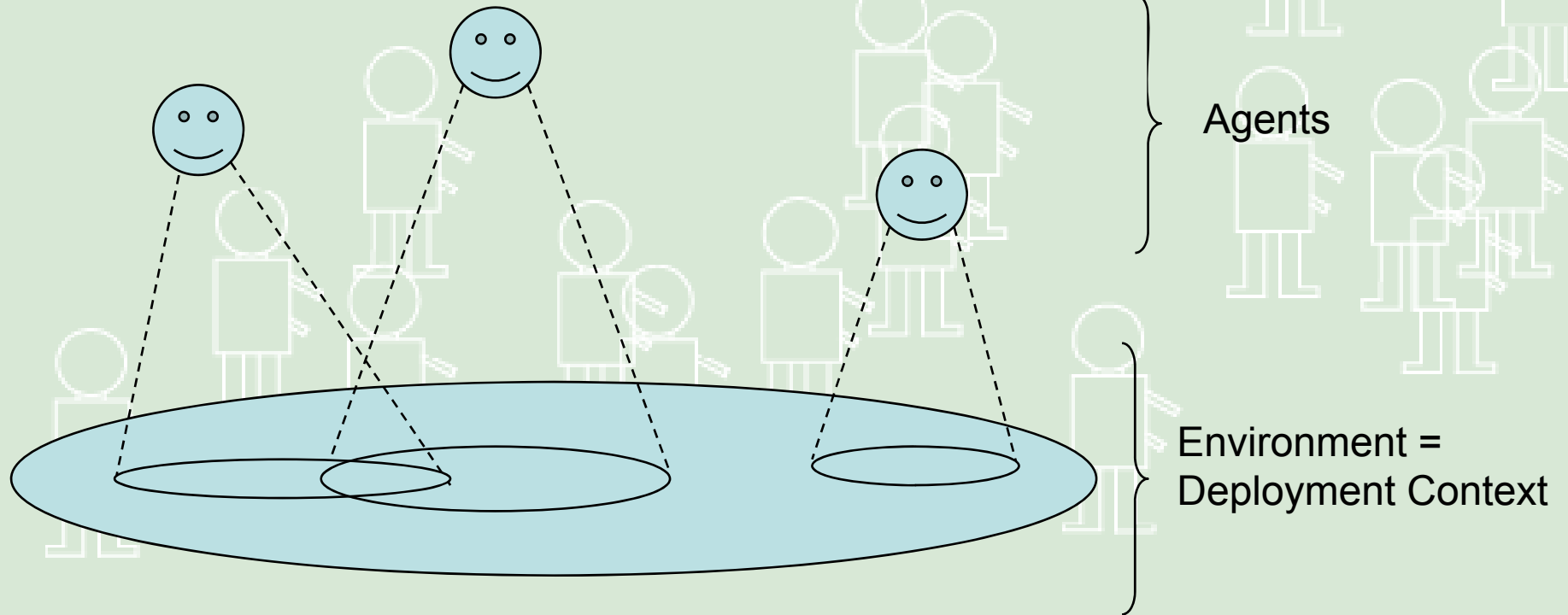
Outline

- Architecture-Centric Software Engineering
- Reference Architecture for Situated Multiagent Systems
 - Background
 - Environment as a First-Class Design Abstraction
 - Advanced Mechanisms for Adaptability
- Conclusions

Reference Architecture

Environment as a First-Class Design Abstraction

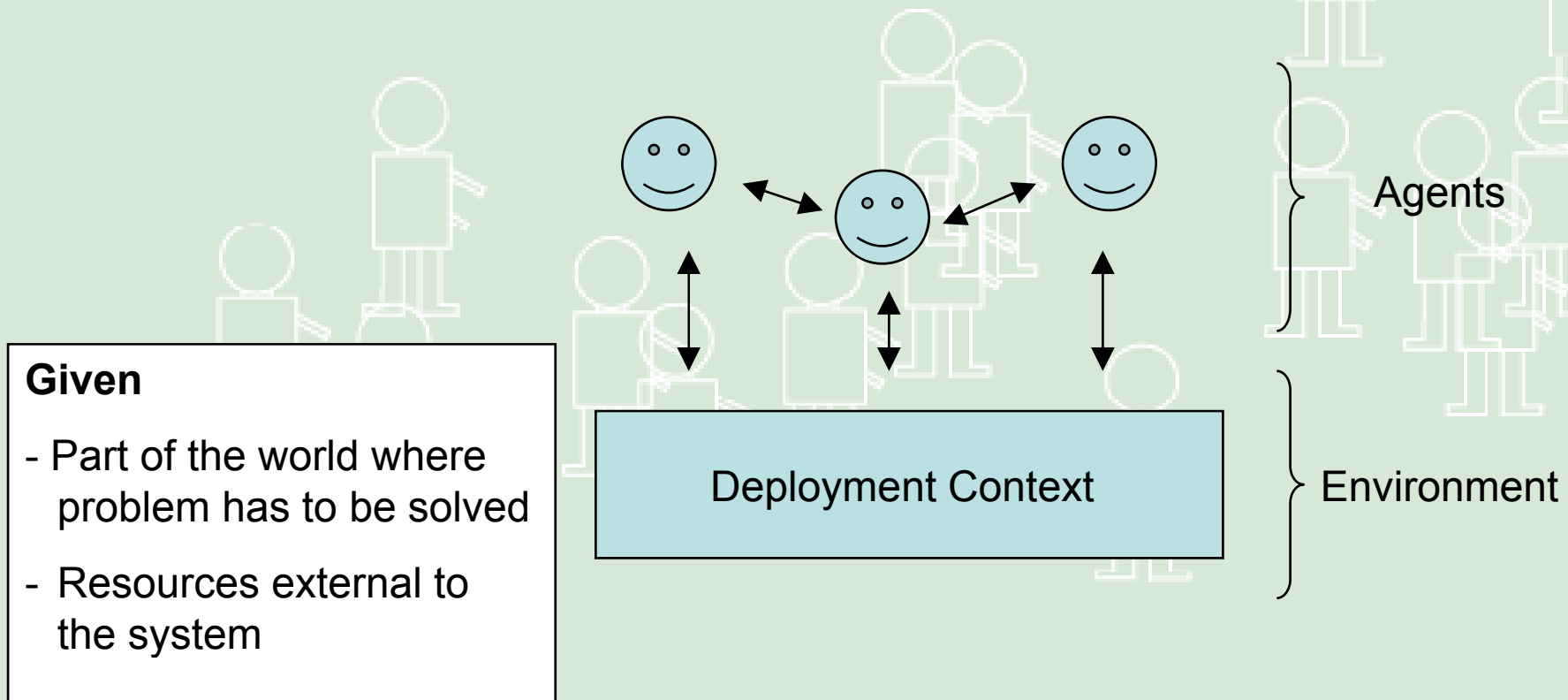
Common perspective on a multiagent system



Reference Architecture

Environment as a First-Class Design Abstraction

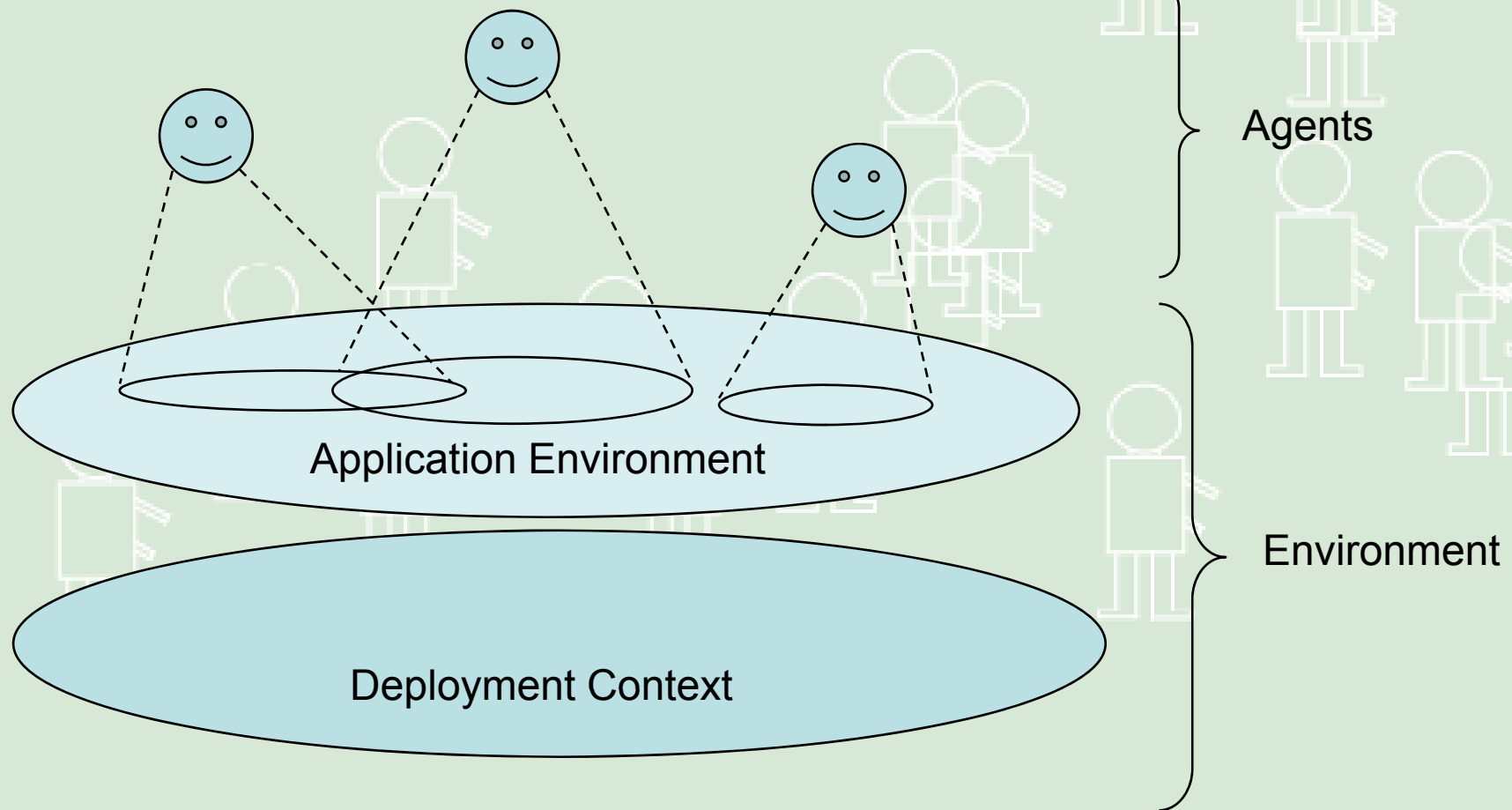
Common perspective on a multiagent system



Reference Architecture

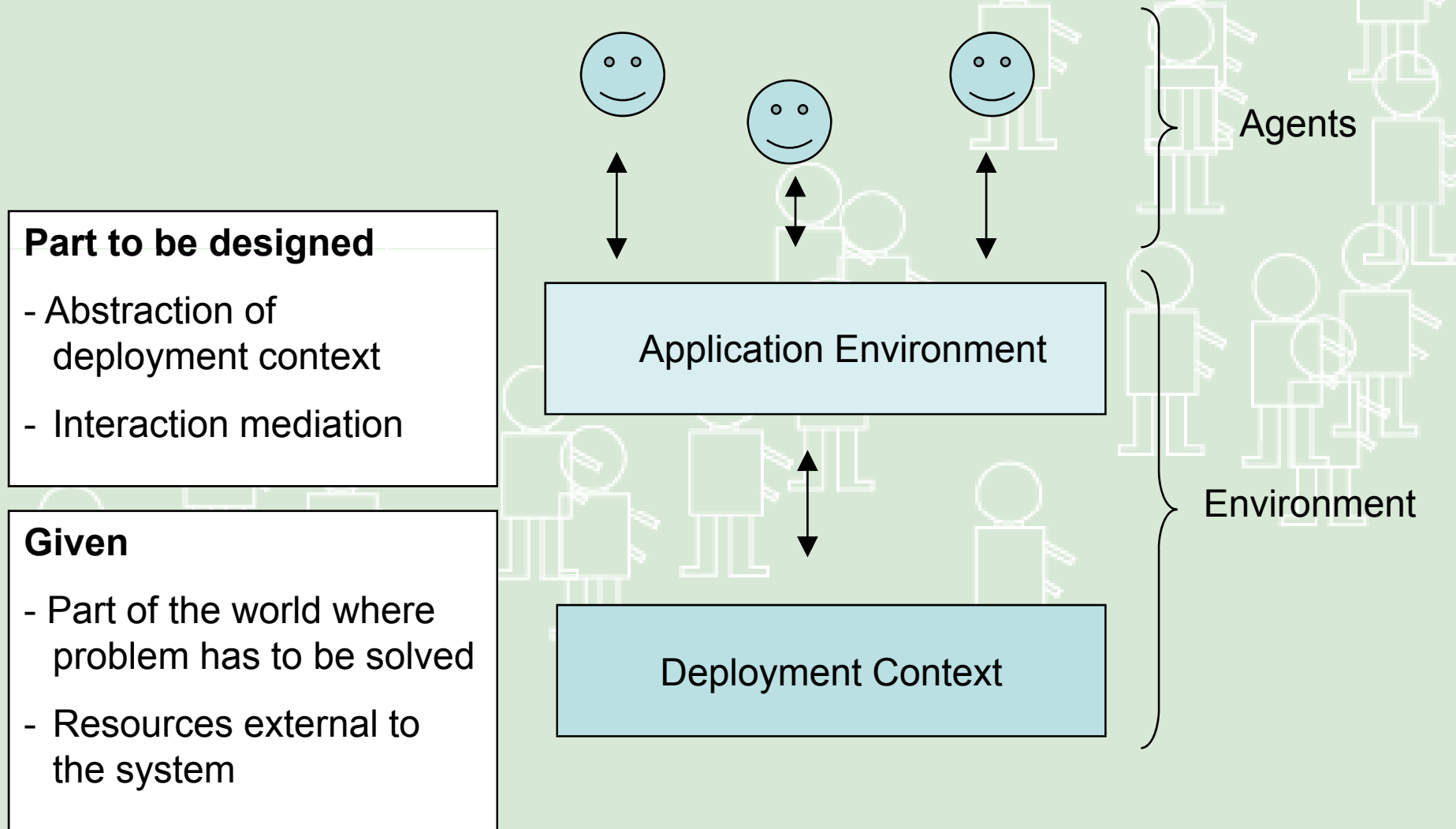
Environment as a First-Class Design Abstraction

Application environment as a design abstraction

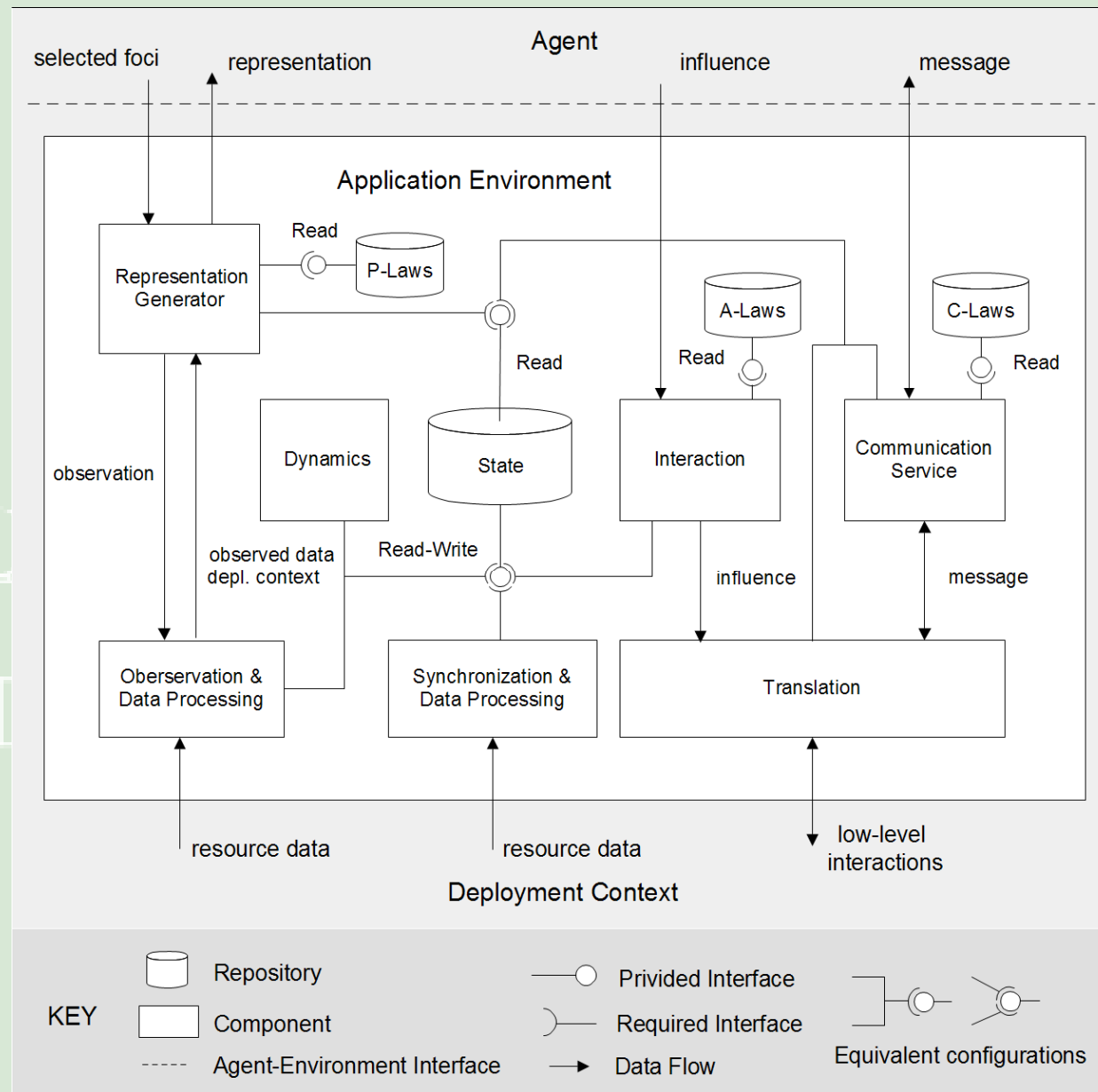


Reference Architecture

Environment as a First-Class Design Abstraction

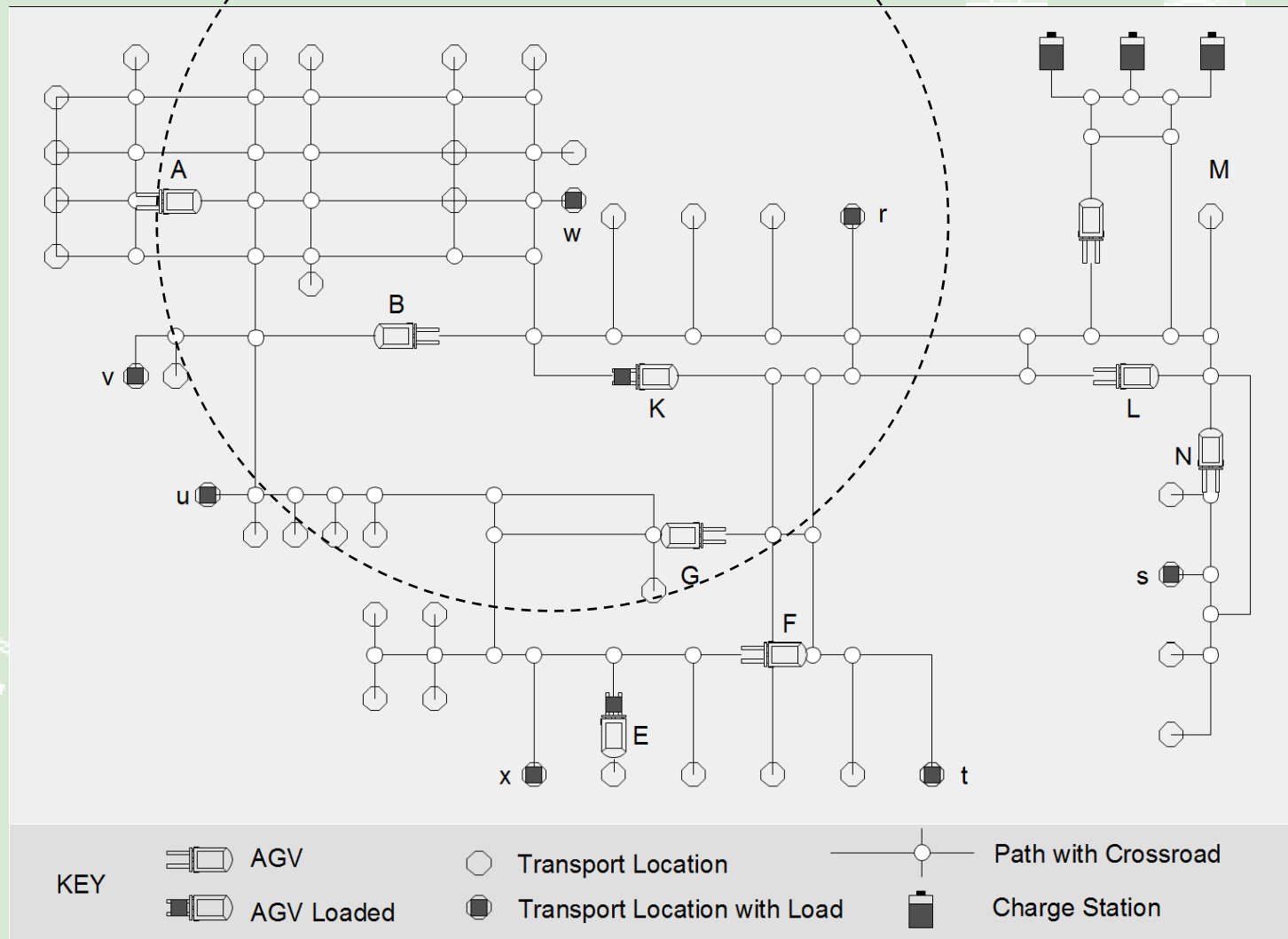


Reference Architecture: Application Environment



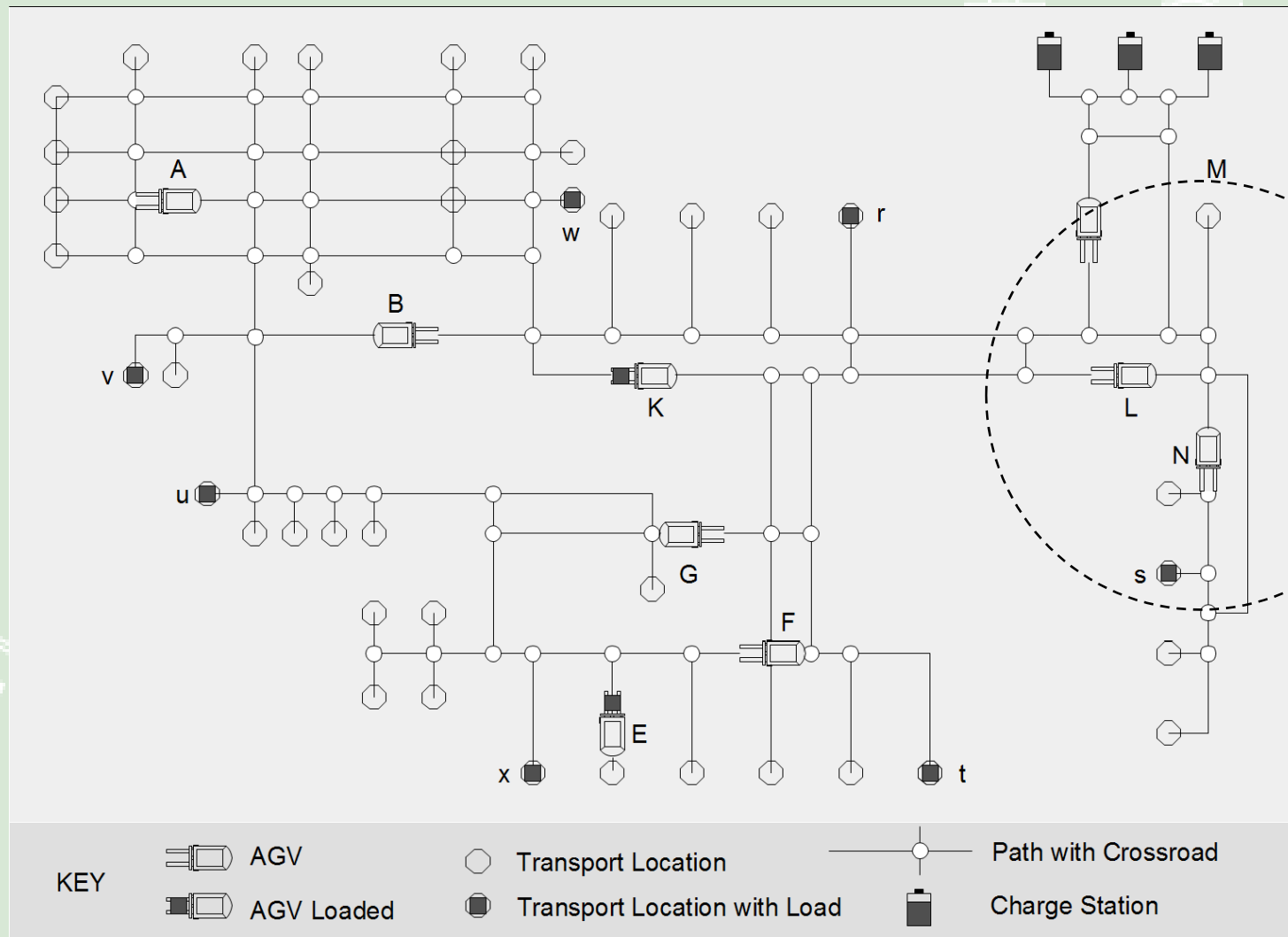
Situated Multiagent Systems

Exploiting the Environment: An Example



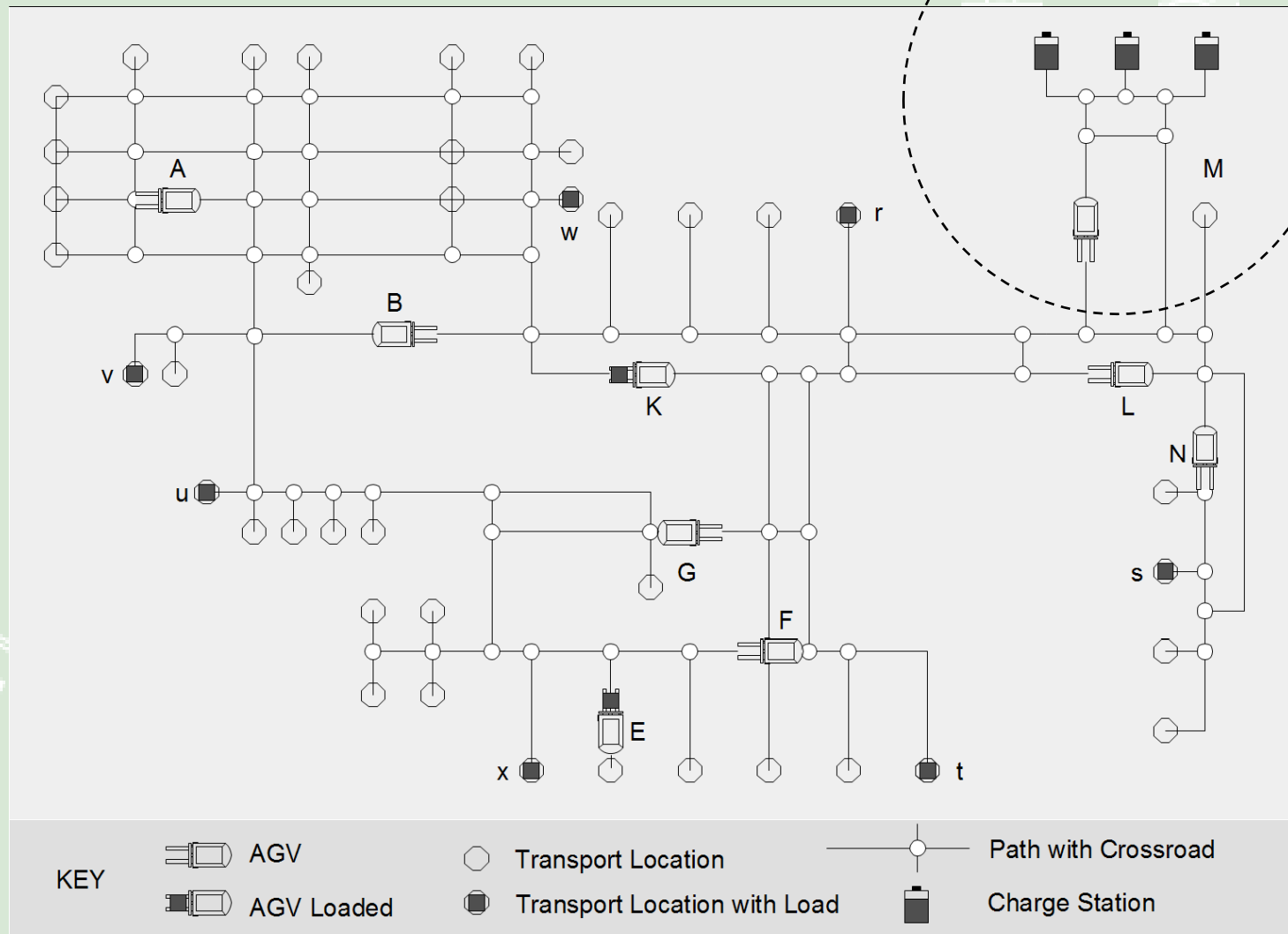
Situated Multiagent Systems

Exploiting the Environment: An Example



Situated Multiagent Systems

Exploiting the Environment: An Example



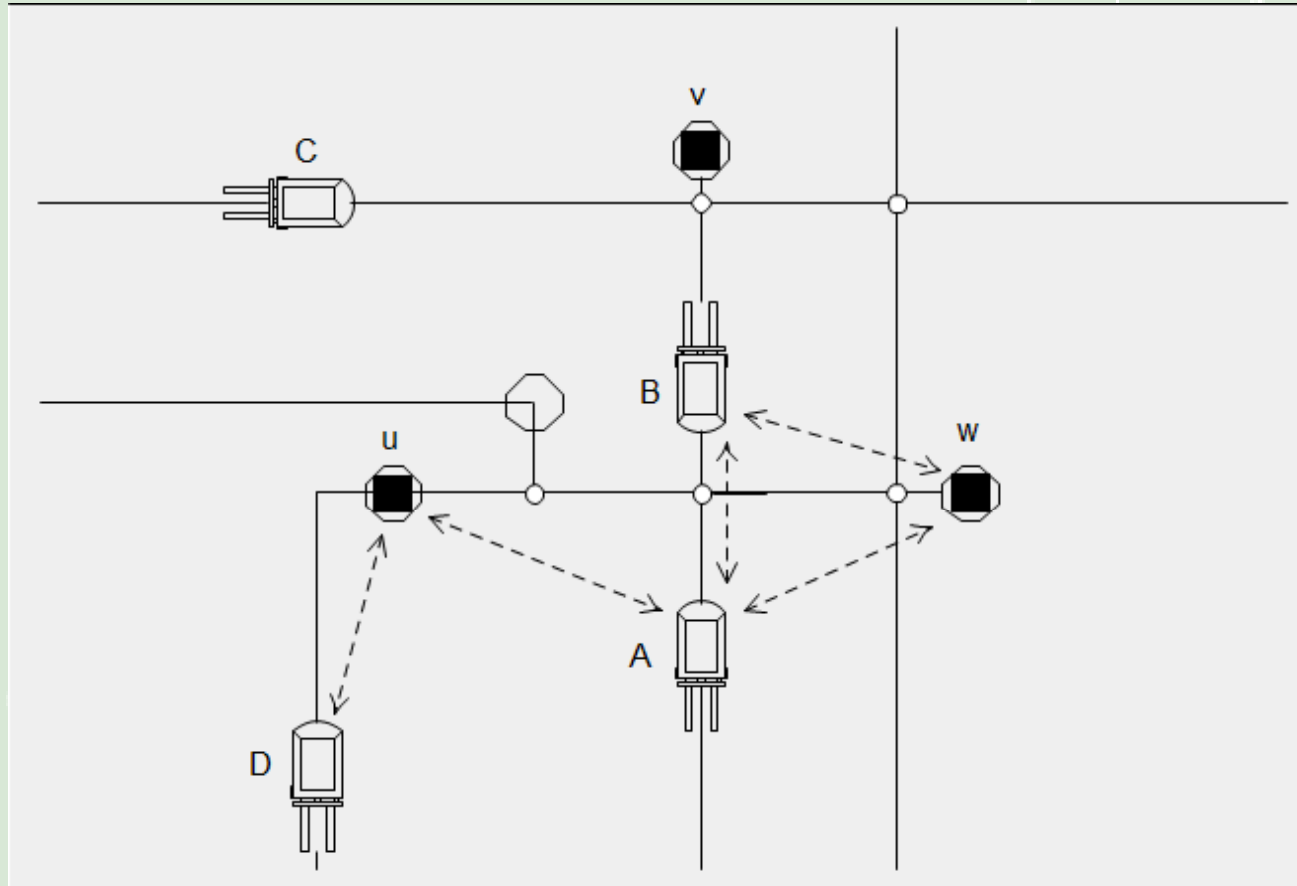
Reference Architecture

Exploiting the environment: an example

- Coordination in AGV transportation system
 - Assignment of tasks
 - Collision avoidance
 - Charging batteries
 -
- How to coordinate agents?
 - “Classic approach”: agents coordinate by exchanging messages
 - Exploit the environment

Reference Architecture

Exploiting the Environment: An Example

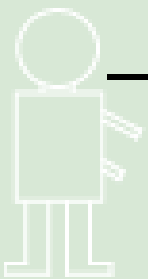


Result: full complexity in the agents

Reference Architecture

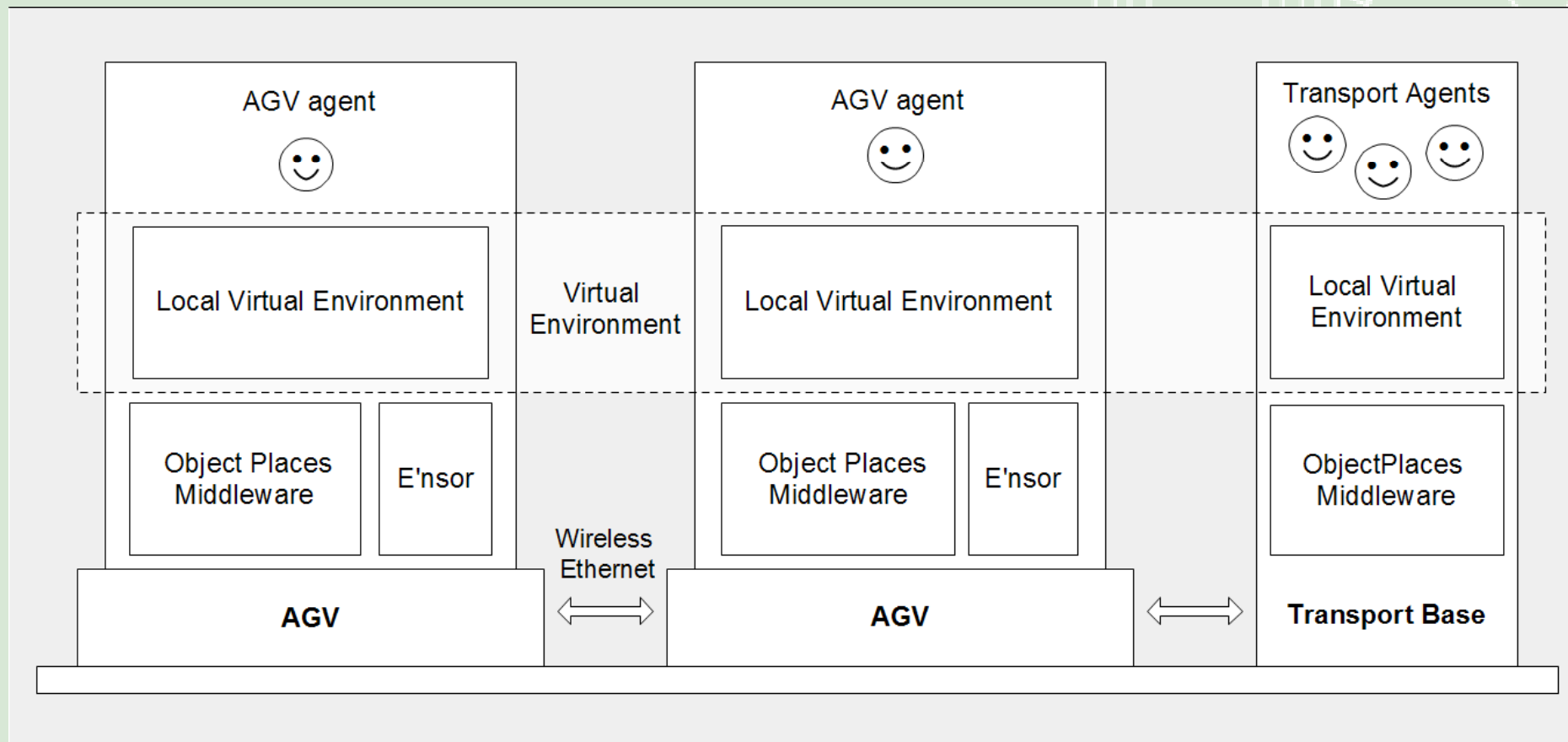
Exploiting the environment: an example

- **Exploit the environment to coordinate**
- However
 - Deployment context restricts how agents can exploit the environment
- We introduced an application environment
 - Enables agents to coordinate through the environment



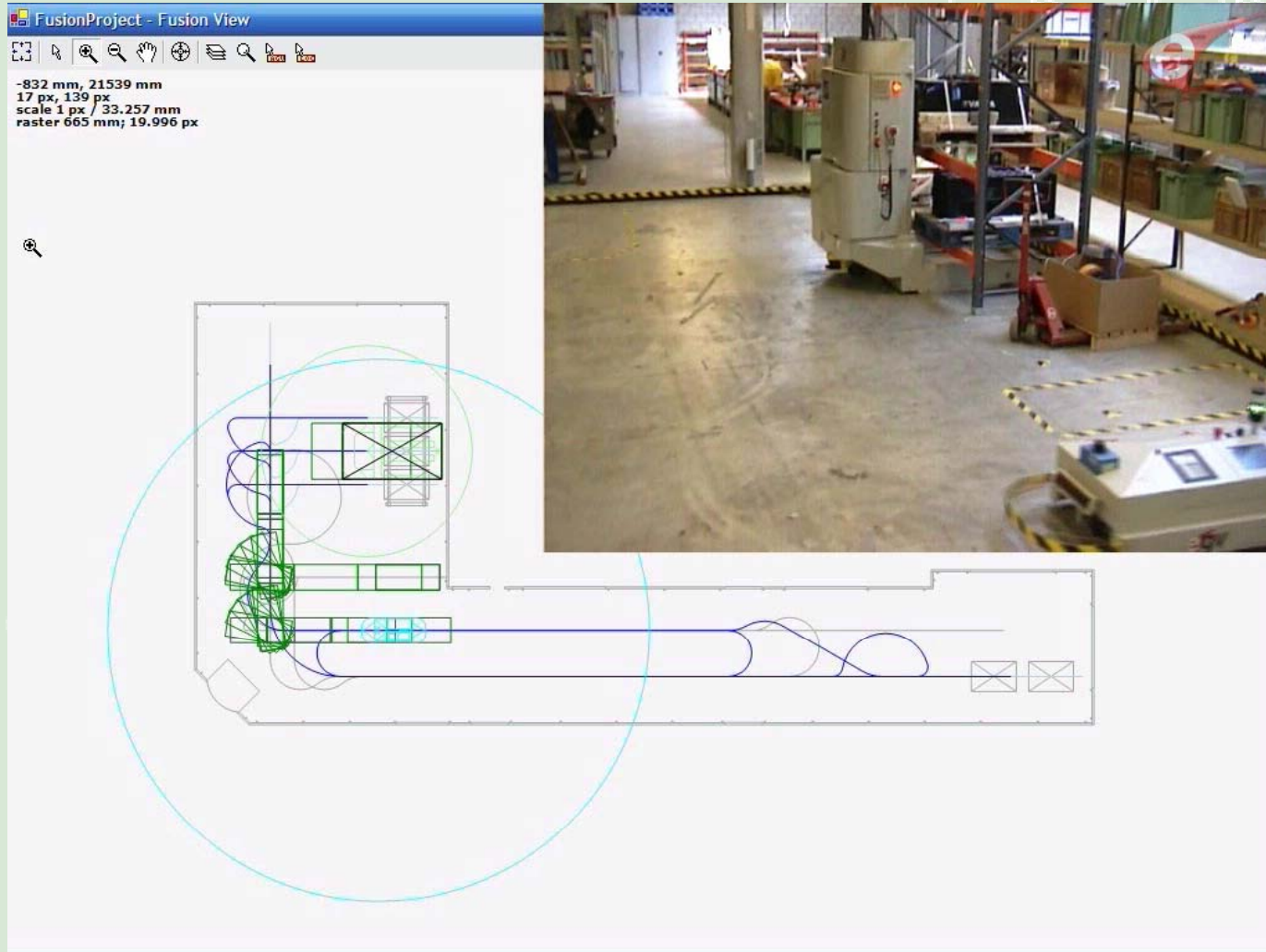
Exploiting the Environment: An Example

Application environment in the AGV transportation system



Exploiting the Environment: An Example

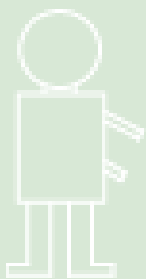
Collision Avoidance



Reference Architecture

Exploiting the environment: an example

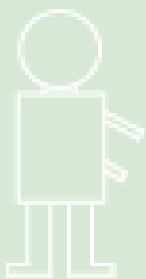
- Advantages of exploiting the environment to coordinate agents (1)
 - Avoid complex agents by splitting up responsibilities
 - Agents are responsible for projecting/removing hulls
 - Application environment is responsible for determining which AGV can drive on



Reference Architecture

Exploiting the environment: an example

- Advantages of exploiting the environment to coordinate agents (2)
 - Flexibility and openness
 - AGVs can dynamically remove hull and select alternative route
 - Application environment takes into account AGVs that enter/leave collision range



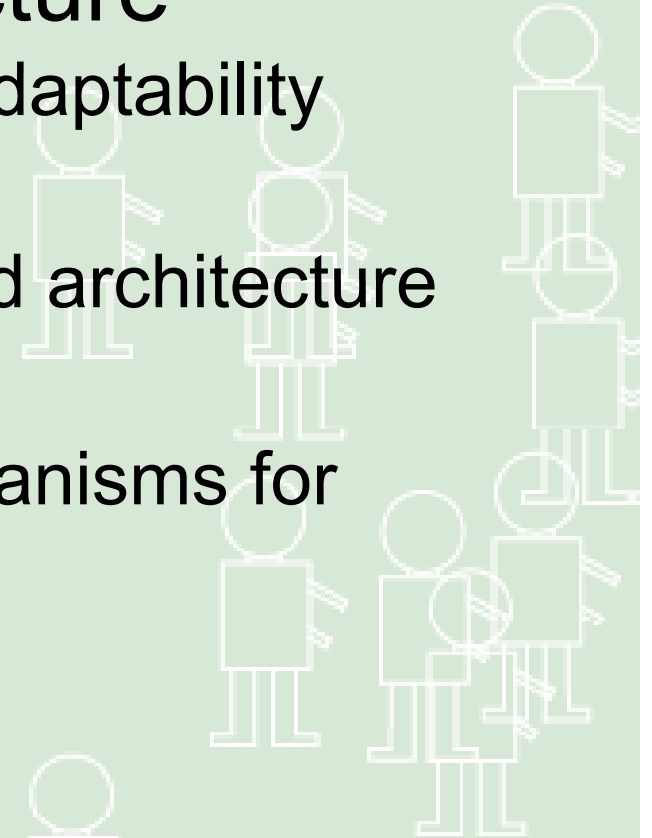
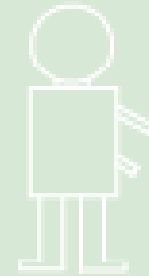
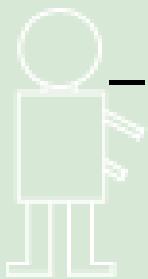
Outline

- Architecture-Centric Software Engineering
- Reference Architecture for Situated Multiagent Systems
 - Background
 - Environment as a First-Class Design Abstraction
 - Advanced Mechanisms for Adaptability
- Conclusions

Reference Architecture

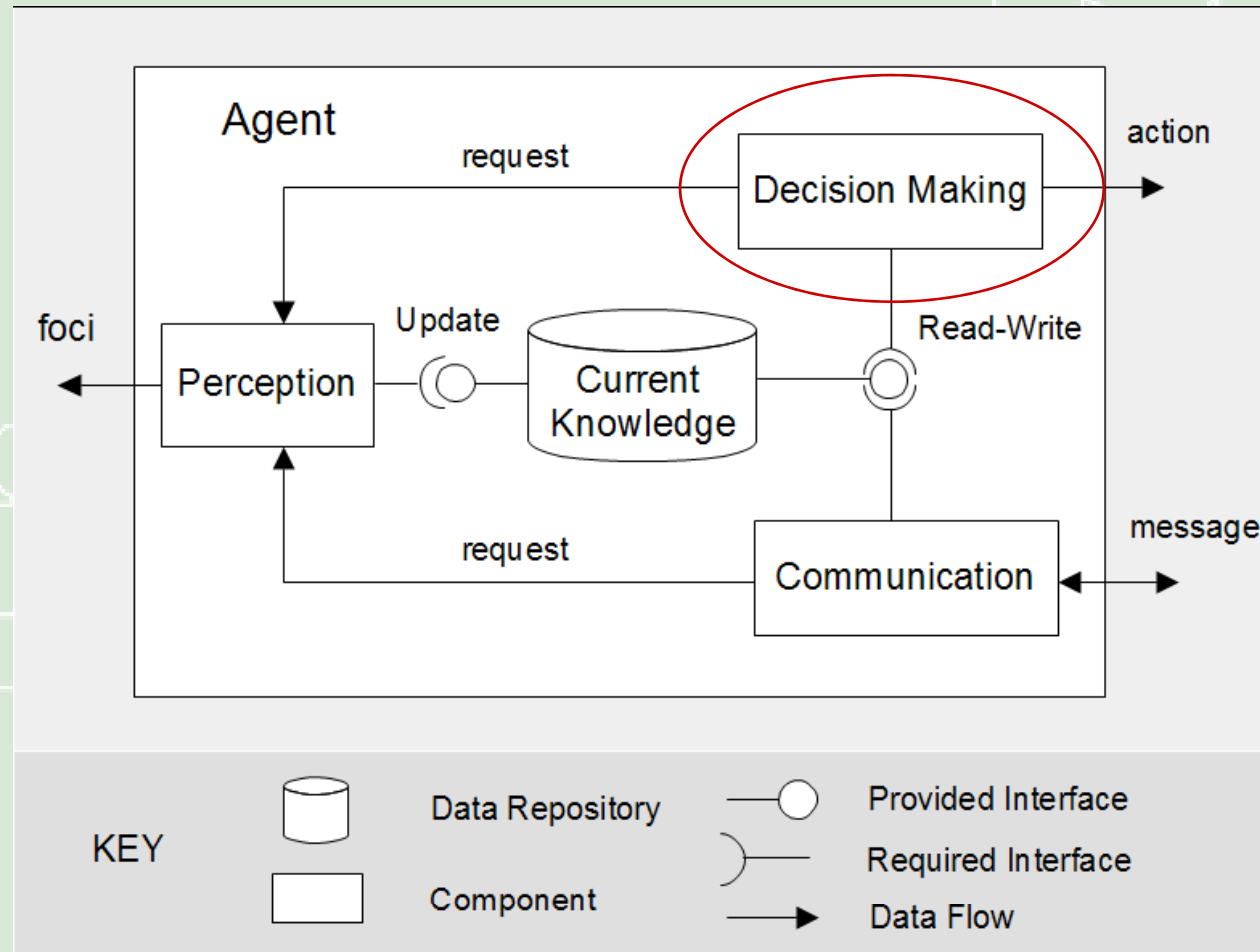
Advanced Mechanisms for Adaptability

- We have developed an integrated architecture for situated agents
- Integrates set of advanced mechanisms for adaptability
 - Selective perception
 - Protocol-based communication
 - Roles and situated commitments



Reference Architecture

Advanced Mechanisms for Adaptability



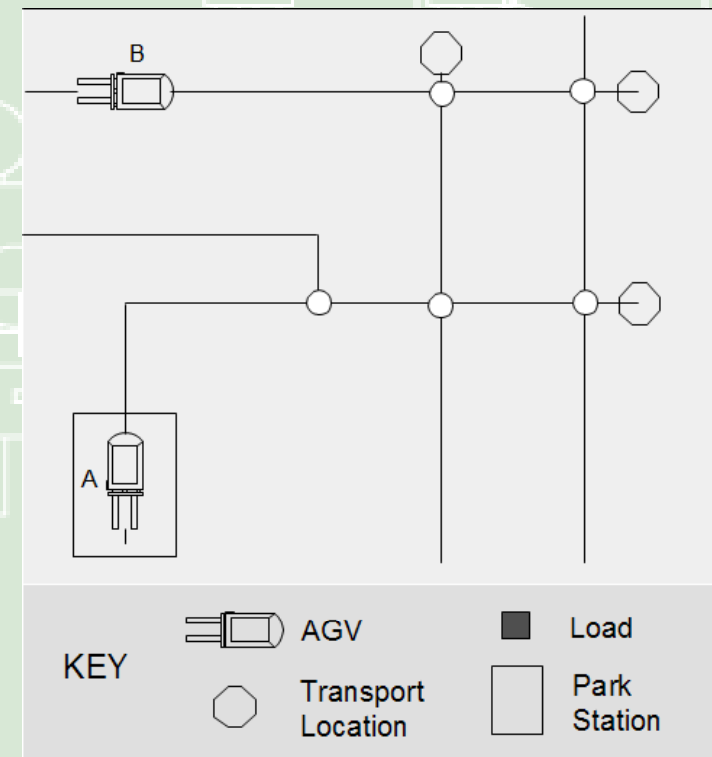
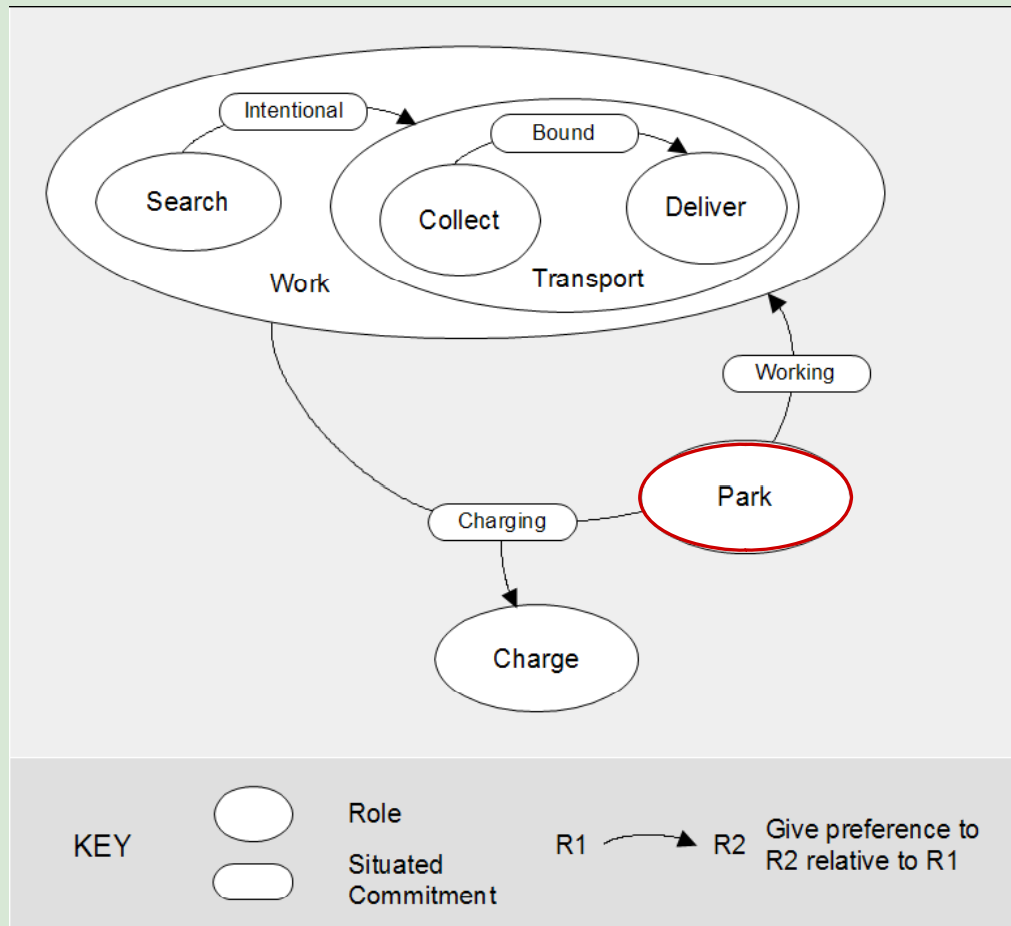
Reference Architecture

Advanced Mechanism for Adaptability

- Roles and situated commitments
 - Endow situated agents with abilities for explicit social interaction
- Role
 - Coherent part of functionality in context of an organization
- Situated Commitment
 - Engagement to give preference to the actions in a particular role
 - Driven by conditions in the environment in which the agents are situated

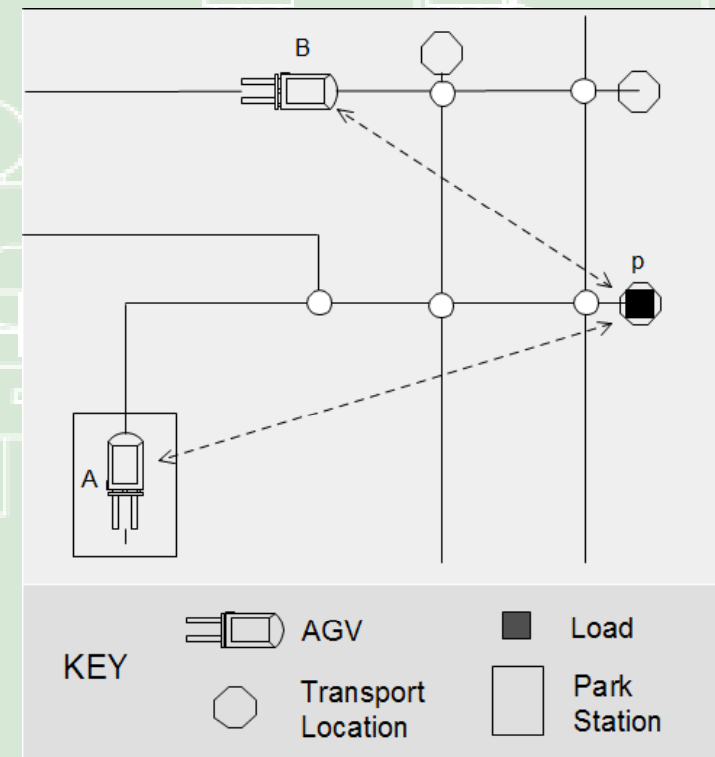
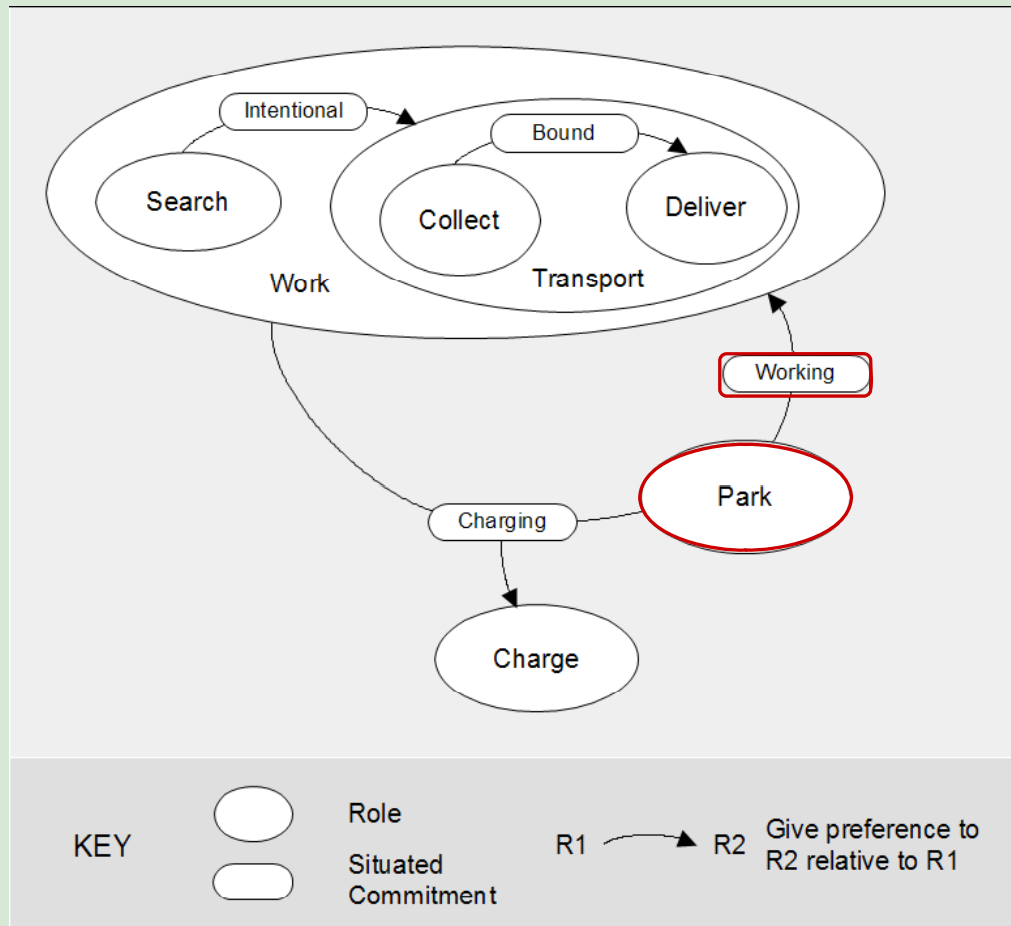
Reference Architecture

Roles and Situated Commitments



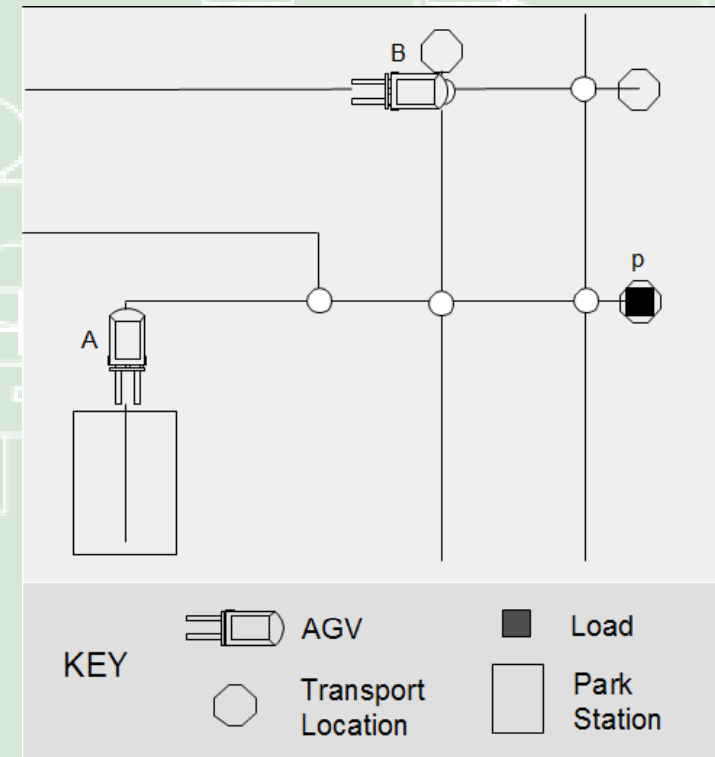
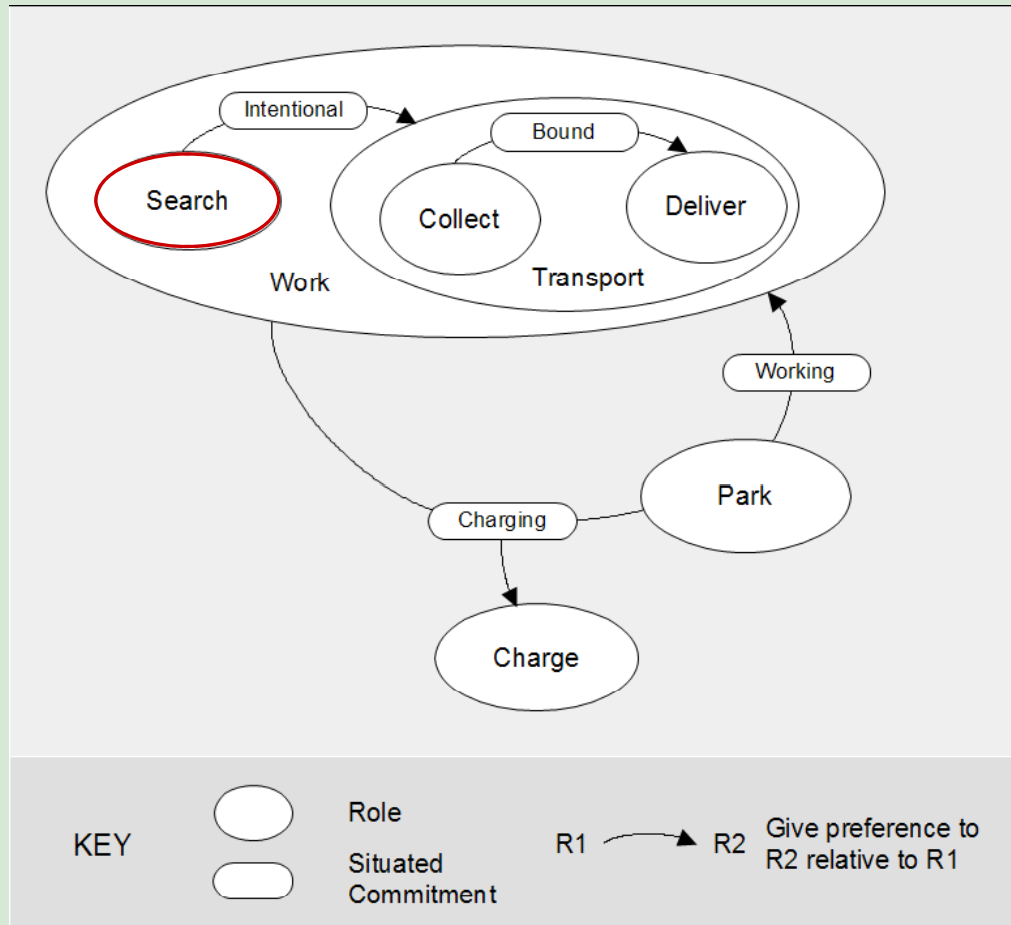
Reference Architecture

Roles and Situated Commitments



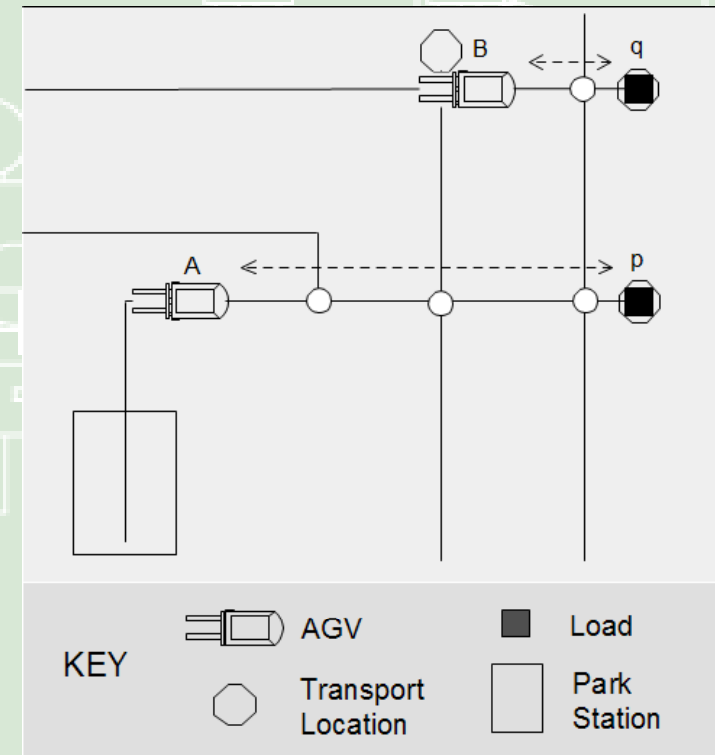
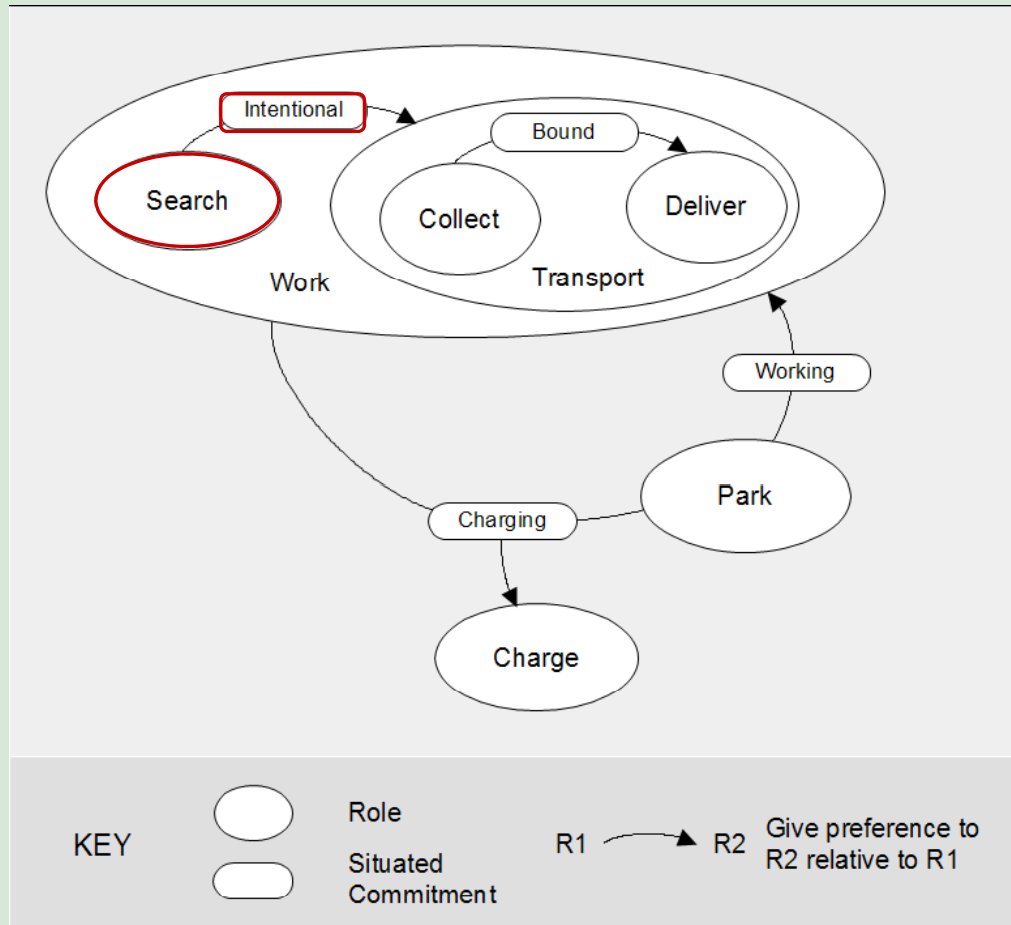
Reference Architecture

Roles and Situated Commitments



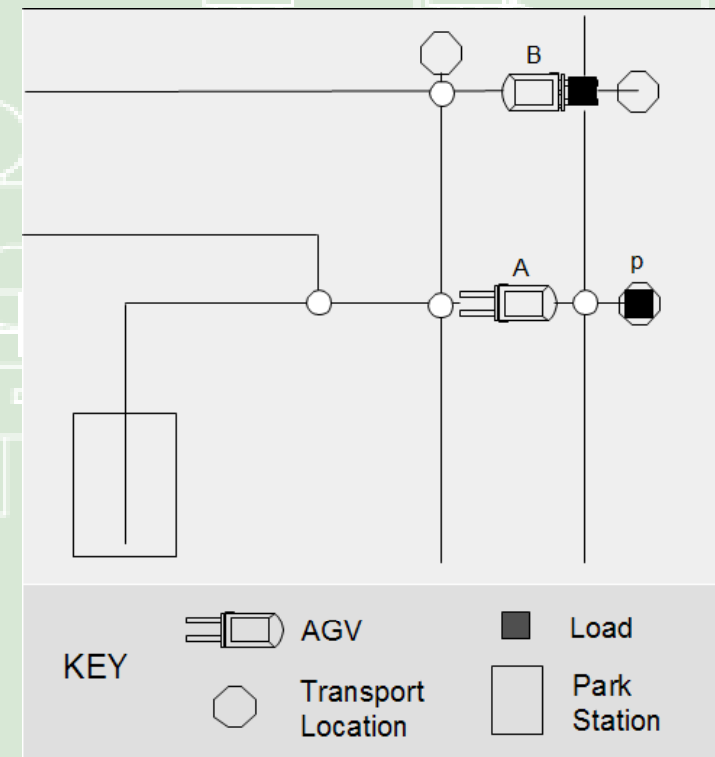
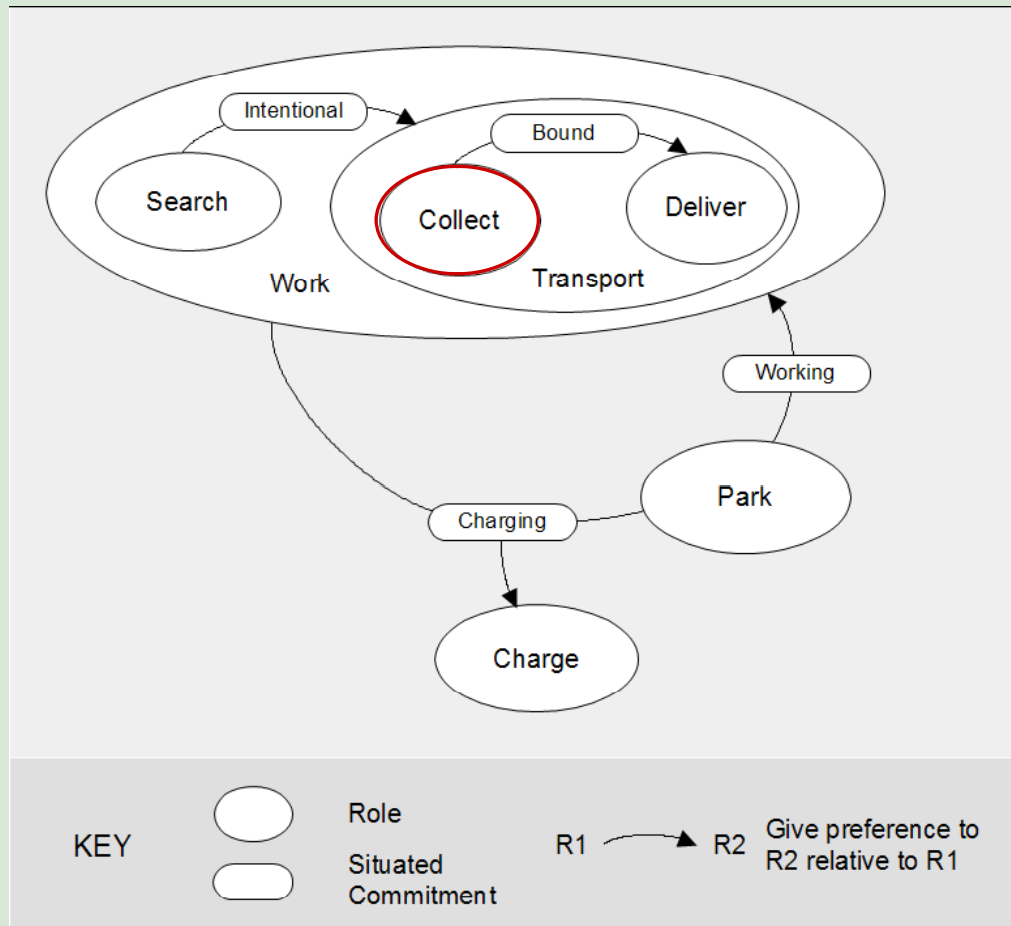
Reference Architecture

Roles and Situated Commitments



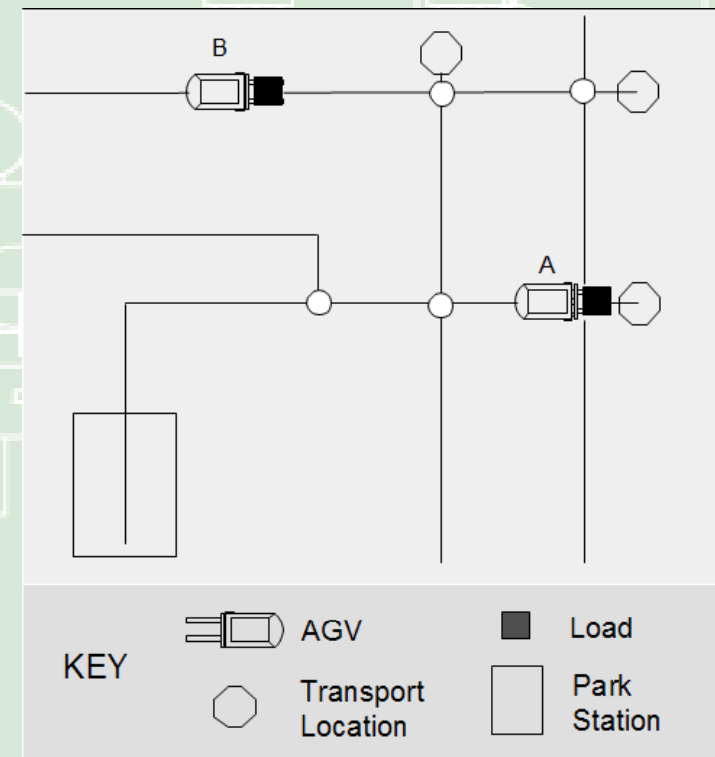
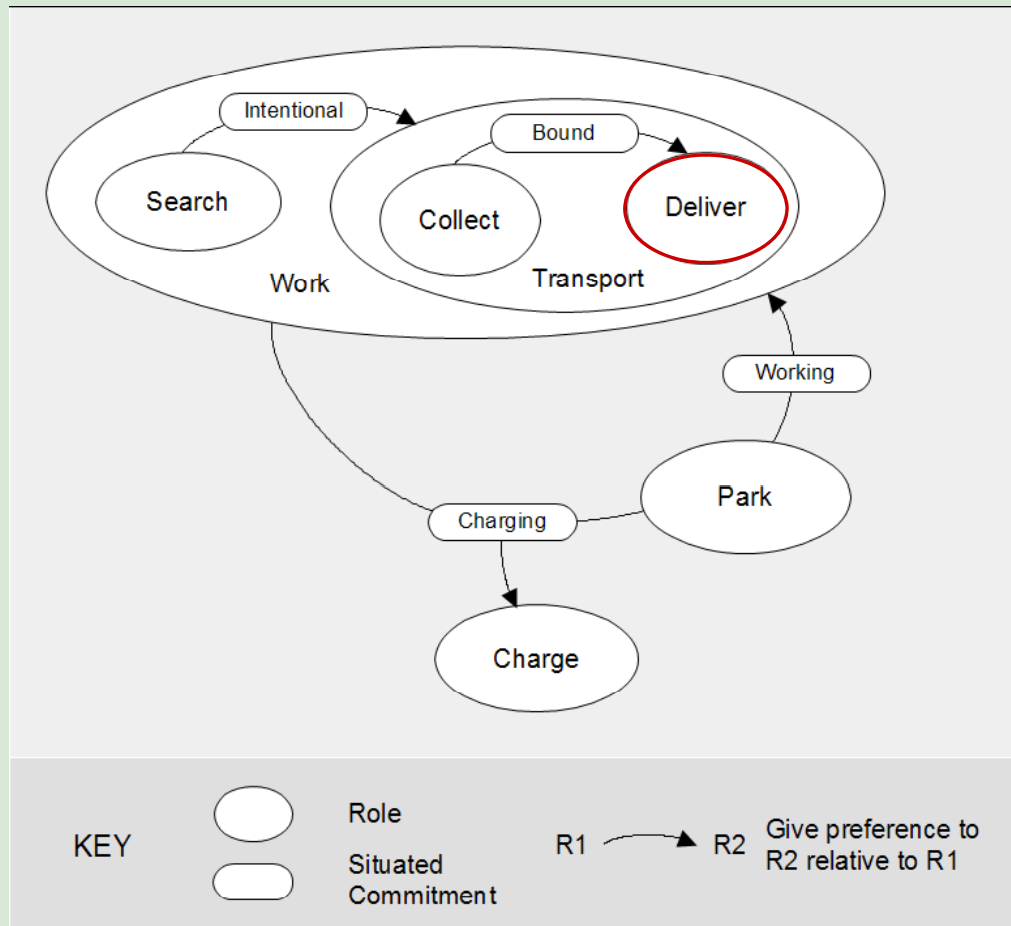
Reference Architecture

Roles and Situated Commitments



Reference Architecture

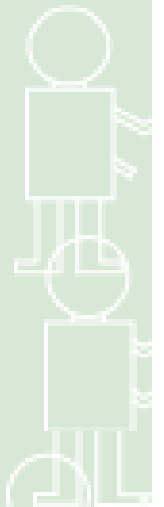
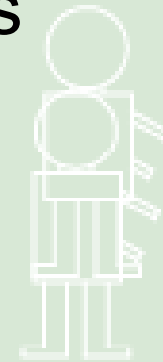
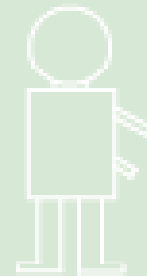
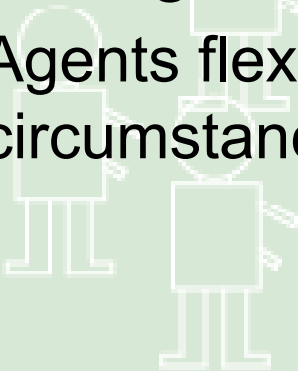
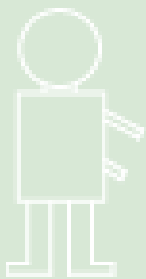
Roles and Situated Commitments



Reference Architecture

Roles and Situated Commitments

- Building blocks for social organization
 - Enable agents to set up collaborations
- Social interaction is driven by the context in which agents are situated
 - Agents flexibly adapt their behavior with changing circumstances in the environment

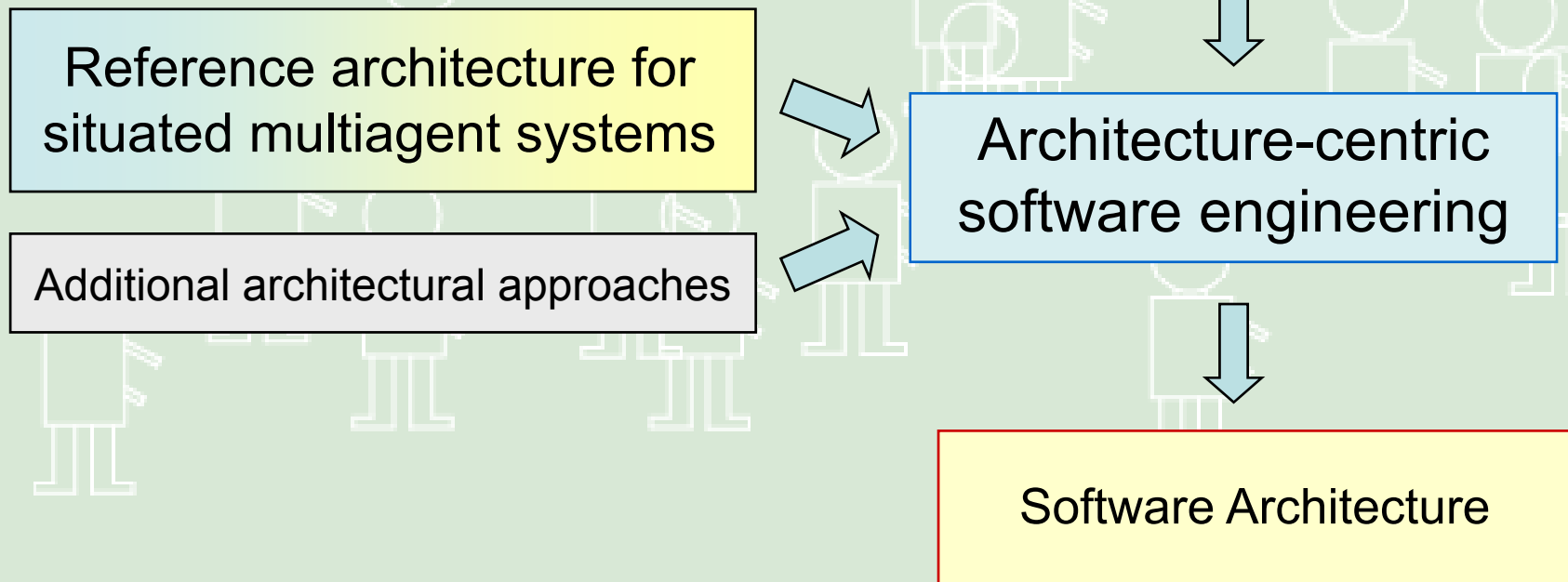


Outline

- Architecture-Centric Software Engineering
- Reference Architecture for Situated Multiagent Systems
- **Conclusions**
 - Summary of Contributions
 - Lessons Learned for Applying Agents in Industry
 - Future Work

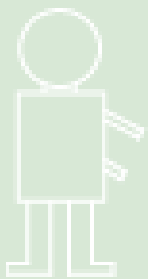
Contributions

- Promising perspective on software engineering with multiagent systems



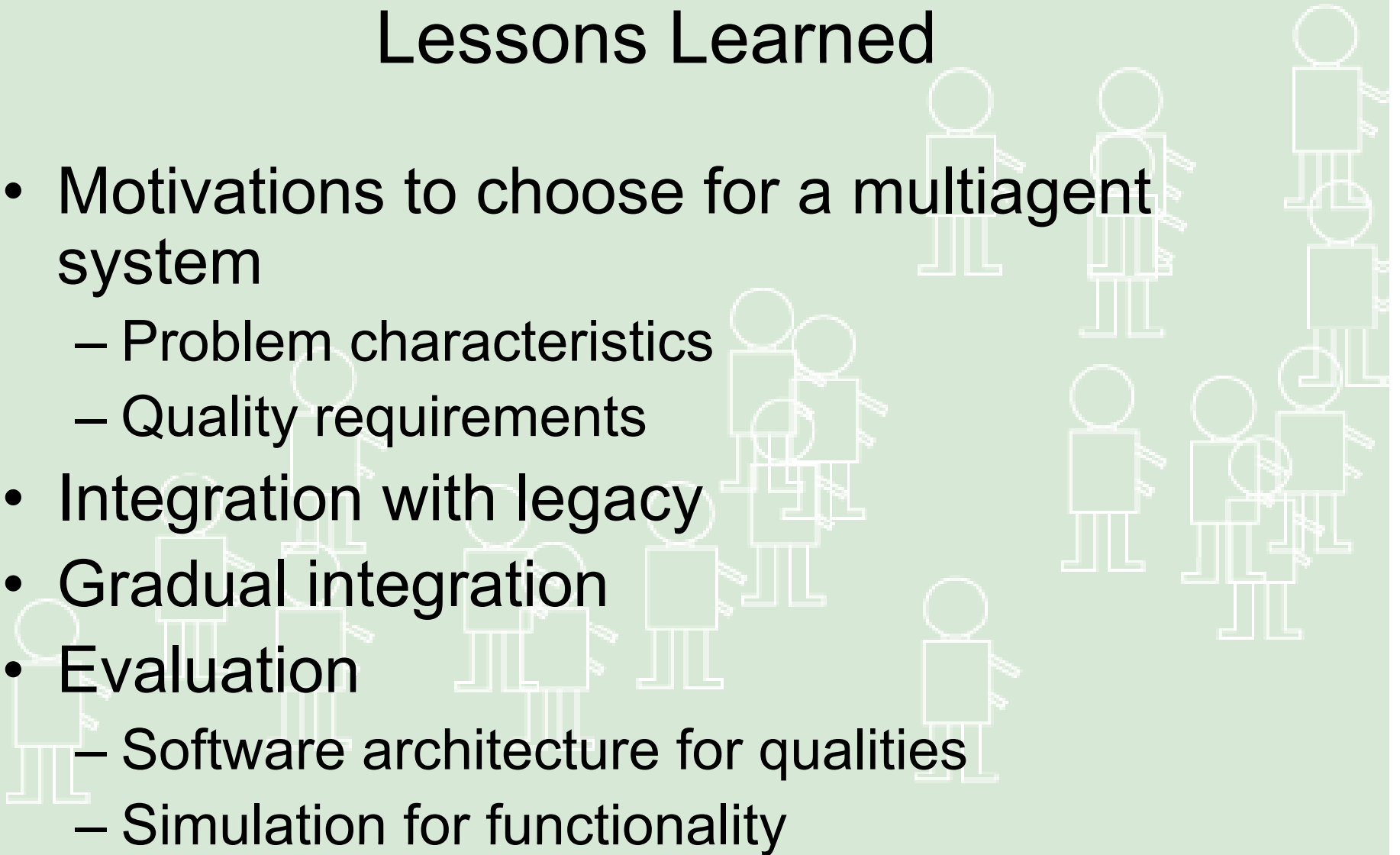
Contributions

- Reference architecture for situated multiagent systems
 - Environment as first-class design abstraction
 - Set of advanced mechanisms for adaptability
 - Selective perception
 - Roles and situated commitments
 - Protocol-based communication



Lessons Learned

- Motivations to choose for a multiagent system
 - Problem characteristics
 - Quality requirements
- Integration with legacy
- Gradual integration
- Evaluation
 - Software architecture for qualities
 - Simulation for functionality



Future Research

- Disciplined approach for architectural design with a reference architecture
- Connection software architecture and multiagent systems
 - Robustness, scalability
 - Patterns for distribution
 - Crosscutting concerns
 - Scientific foundation for verifying global behavior

Closing Reflection

- Situated agency has been studied and applied for two decades
- Reference architecture reifies our expertise that is founded on rich tradition
- Reference architecture demonstrates how MAS can be integrated with mainstream software engineering practice

→ A key for industrial adoption of multiagent systems